

Documentation on Description of API endpoints are which user stories they are linked with

User Management APIs

1. POST /user/create

Purpose: Registers a new user into the system. This user could be a senior citizen, guardian, or admin, based on the is_admin field.

Request Body:

```
{  
  
  "name": "string",  
  
  "email": "string",  
  
  "age": int,  
  
  "gender": "string",  
  
  "phone_number": "string",  
  
  "is_admin": bool  
}
```

Behavior:

- Validates that both email and phone number are unique.
- Ensures all required fields are provided.
- Saves the user details in the database.

Response:

```
{"success": true, "message": "User created successfully"}
```

User Story Mapping:

- As a senior citizen or guardian, I want to register to gain access to personalized features.
- As an admin, I want to create user accounts for new participants.

2. GET /user/get/{user_id}

Purpose: Fetches detailed profile information for a specific user by their ID.

Behavior:

- Looks up the user in the database.
- Returns the full user profile (name, email, age, gender, phone number, role).

User Story Mapping:

- As a user, I want to view my own profile.
- As an admin, I need access to user information to support and manage accounts.

3. PUT /user/update

Purpose: Updates information for an existing user.

Request Body:

```
{  
  "id": int,  
  "name": "string",  
  "email": "string",  
  "age": int,  
  "gender": "string",  
  "phone_number": "string"
```

}

Behavior:

- Verifies that the user exists.
- Updates only the fields provided in the request.

User Story Mapping:

- As a user, I want to correct or update outdated details (e.g., new phone number).
- As an admin, I may assist users in updating their records.

4. DELETE /user/delete/{user_id}

Purpose: Deletes a user permanently from the system.

Behavior:

- Checks if the user ID exists.
- Removes all associated data securely.

User Story Mapping:

- As an admin, I want to maintain data cleanliness by removing inactive or duplicate users.

Reminder Management APIs

5. POST /reminder/create

Purpose: Allows a user to set automated reminders for health, medications, or appointments.

Request Body:

{

```
"user_id": int,  
"title": "string",  
"frequency": "daily | weekly",  
"time": "HH:MM"  
}
```

Behavior:

- Validates time and frequency format.
- Links the reminder to the specified user.

User Story Mapping:

- As a senior citizen, I want to create reminders to take medication or attend events so I don't forget important tasks.

6. GET /reminder/get/{reminder_id}

Purpose: Retrieves details of a reminder by its unique ID.

Behavior:

- Returns the title, time, and frequency of the reminder.

User Story Mapping:

- As a user, I want to review the reminders I have created so I can plan my day.

7. PUT /reminder/update

Purpose: Edits an existing reminder.

Request Body:

```
{  
  "id": int,  
  "title": "string",  
  "frequency": "daily | weekly",  
  "time": "HH:MM"  
}
```

Behavior:

- Checks if the reminder exists.
- Updates the relevant fields only.

User Story Mapping:

- As a user, I want to adjust reminders to match changes in my schedule (e.g., new appointment time).

8. DELETE /reminder/delete/{reminder_id}

Purpose: Removes a reminder completely.

Behavior:

- Deletes the reminder by ID.

User Story Mapping:

- As a user, I want to clear reminders that are no longer relevant to my routine.

Messaging APIs

9. POST /message/create

Purpose: Enables users to send messages (e.g., from senior citizen to guardian).

Request Body:

```
{  
  "sender_id": int,  
  "recipient_id": int,  
  "subject": "string",  
  "content": "string"  
}
```

Backend Details:

- Stores metadata (sender, recipient, timestamp) in MySQL.
- Stores full message body in MongoDB for flexible querying.

User Story Mapping:

- As a senior citizen, I want to communicate easily with my guardian.
- As a guardian, I want to check in and offer help to my dependent.

10. **GET** /message/get/{message_id}

Purpose: Fetches a complete message (metadata and body).

User Story Mapping:

- As a user, I want to view my received or sent messages for reference.

11. **DELETE** /message/delete/{message_id}

Purpose: Deletes a specific message.

Behavior:

- Removes the message entry from both MongoDB and MySQL.

User Story Mapping:

- As a user, I want to clean up old or irrelevant conversations.

Content Management APIs**12. POST /content/create**

Purpose: Uploads educational or useful content (video, PDF, or image) for users.

Request Body:

```
{  
  "title": "string",  
  "type": "pdf | video | image",  
  "file_url": "string"  
}
```

Behavior:

- Validates the type and format.
- Stores the reference to the content in the database.

User Story Mapping:

- As an admin, I want to share helpful health tips, tutorials, or activities.
- As a senior citizen, I want access to meaningful, age-appropriate resources.

13. GET /content/get/{content_id}

Purpose: Retrieves a specific content entry for the user to view.

User Story Mapping:

- As a user, I want to browse or view uploaded content (e.g., how-to videos).

14. **DELETE** /content/delete/{content_id}

Purpose: Deletes existing content.

Behavior:

- Removes file metadata and ensures the file is no longer accessible.

User Story Mapping:

- As an admin, I want to manage the quality of available content.

User Registration & Authentication APIs

15. **POST** /register

Purpose :

This API allows users (typically senior citizens or guardians) to create an account. It ensures no duplicate email or phone number is used. If the information is unique, it stores the data in the SQL database and confirms successful registration.

Request Body (JSON):

json

{

"name": "John Doe",

"email": "john@example.com",

"phone_number": "9876543210",

"age": 65,

```
"gender": "Male",  
"is_admin": false  
}
```

Behavior :

- Registers a new user by checking if the email or phone number is already used.
- Adds the new user to the relational database.

User story Mapping:

- Supports the user story: “As a senior citizen, I want to register with my basic details so that I can use the platform.”

16. POST /login/send_otp

Purpose :

Instead of remembering passwords, users receive a one-time password (OTP) for logging in securely. Redis is used for fast and temporary OTP storage.

Request Body (JSON):

```
json  
  
{  
  
  "email": "john@example.com"  
  
}
```

Behavior:

- Generates a 6-digit OTP.

- Sends it to the provided email using the configured email utility.
- Stores OTP temporarily in Redis for 5 minutes (300 seconds).

User Story Mapping:

- Supports the user story: “As a senior citizen, I want to log in using OTP because I might forget passwords.”

17. POST /login/verify_otp

Purpose:

This endpoint finalizes the login flow. Upon correct OTP verification, it returns the user’s role. Role distinction helps in rendering different dashboards for admins or normal users.

Request Body (JSON):

json

```
{  
  
  "email": "john@example.com",  
  
  "otp": "123456"  
}
```

Behavior :

- Verifies if the OTP entered matches the one stored in Redis.
- Deletes OTP after verification.
- Identifies the user and determines their role (admin or user).

User story Mapping:

- Supports the story: “As a user, I want secure OTP login so that I can access the dashboard based on my role.”